



TILERA[®]

MULTICORE DEVELOPMENT ENVIRONMENT

TILE PROCESSOR[™] APPLICATION BINARY INTERFACE

RELEASE 4.3.4.193953
DOC. No. UG513
OCTOBER 26, 2015
TILERA CORPORATION

© Copyright 2014 Tiler Corporation. All rights reserved. Printed in the United States of America.

The information contained in this document is the property of Tiler Corporation. It has been released under Non-Disclosure Agreement. Any unauthorized review, use, disclosure or distribution is strictly prohibited.

No part of this book may be reproduced, stored in a retrieval system, or transmitted in any form or by any means, electronic, mechanical, photocopying, recording, or otherwise, except as may be expressly permitted by the applicable copyright statutes or in writing by the Publisher.

The following are registered trademarks of Tiler Corporation: Tiler and the Tiler logo. The following are trademarks of Tiler Corporation: Embedding Multicore, The Multicore Company, Tile Processor, TILE Architecture, TILE64, TILEPro, TILEPro36, TILEPro64, TILEExpress, TILEExpress-64, TILEExpressPro-64, TILEExpress-20G, TILEExpressPro-20G, TILEExpressPro-22G, iMesh, TileDirect, TILEmpower, TILEmpower-Gx, TILEncore, TILEncorePro, TILEncore-Gx, TILE-Gx, TILE-Gx16, TILE-Gx36, TILE-Gx64, TILE-Gx100, TILE-Gx3000, TILE-Gx5000, TILE-Gx8000, DDC (Dynamic Distributed Cache), Multicore Development Environment, Gentle Slope Programming, iLib, TMC (Tiler Multicore Components), hardwall, Zero Overhead Linux (ZOL), MiCA (Multicore iMesh Coprocessing Accelerator), and mPIPE (multicore Programmable Intelligent Packet Engine). All other trademarks and/or registered trademarks are the property of their respective owners.

Third-party software: The Tiler IDE makes use of the BeanShell scripting library. Source code for the BeanShell library can be found at the BeanShell website (<http://www.beanshell.org/developer.html>).

The following is a trademark of Marvell Semiconductor, Inc.: Distributed Switching Architecture (DSA).

This document contains advance information on Tiler products that are in development, sampling or initial production phases. This information and specifications contained herein are subject to change without notice at the discretion of Tiler Corporation.

No license, express or implied by estoppels or otherwise, to any intellectual property is granted by this document. Tiler disclaims any express or implied warranty relating to the sale and/or use of Tiler products, including liability or warranties relating to fitness for a particular purpose, merchantability or infringement of any patent, copyright or other intellectual property right.

Products described in this document are NOT intended for use in medical, life support, or other hazardous uses where malfunction could result in death or bodily injury.

THE INFORMATION CONTAINED IN THIS DOCUMENT IS PROVIDED ON AN “AS IS” BASIS. Tiler assumes no liability for damages arising directly or indirectly from any use of the information contained in this document.

Mon Oct 26 22:22:59 EDT 2015

Tiler Corporation
Information: info@tilera.com
Website: <http://www.tilera.com>

Contents

1	Introduction	1
2	Machine Interface	3
2.1	Instruction Set Architecture	3
2.2	Data Representation	3
2.2.1	Byte Ordering	3
2.2.2	Scalar Types	3
2.2.3	Aggregates and Unions	5
2.3	Function Call Specification	8
2.3.1	Register Usage	8
2.3.2	Stack	8
2.3.3	Argument Passing	10
2.3.4	Variadic Functions	10
2.3.5	Return Values	10
2.4	Binary Image Format	10
3	System Interface	11
3.1	System Calls	11

CONTENTS

Chapter 1

Introduction

The TILE-Gx Processor provides a programmer with a rectangular grid of tiled processors. Each processor supports a 48-bit virtual address space, mapped onto 64-bit physical addresses. This document describes the Application Binary Interface, or ABI, for programs running on Tile Processors.

The ABI specifies how an application, stored in binary form, will be executed on the machine. It specifies the function calling convention, how an application is stored on disk, how data structures are represented in memory, and how programs interface with the operating system.

Chapter 2

Machine Interface

This chapter describes the binary formats and calling conventions used with the TILE-Gx Processor ABI. This information is particularly important to compiler writers and assembly programmers, as it specifies how structures, array, and unions are formed and how functions are called.

2.1 Instruction Set Architecture

ABI compliant programs use the TILE-Gx Processor ISA, as specified in the TILE-Gx Instruction Set Architecture Specification.

2.2 Data Representation

The Tile Processor Architecture is little-endian with strict alignment requirements. This section describes how data structures should be arranged in memory to meet these requirements in a standardized fashion. Arranging data as described below will also guarantee inter-operation between binaries compiled in different environments.

2.2.1 Byte Ordering

The Tile Processor Architecture is little-endian; the least significant byte in a multi-byte data item is stored at the lowest address. Figure 2.1 illustrates how data bytes are ordered in different sized data types.

2.2.2 Scalar Types

Table 2.1 defines the mapping from ANSI C types to Tile Processor data types. Long integers occupy eight bytes and floating point types must be stored in standard IEEE format.

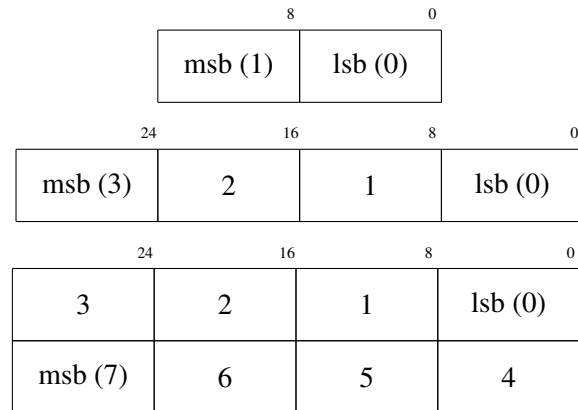


Figure 2.1: Halfword, word, and doubleword byte ordering. Numbers indicate the byte offset in memory.

C type	sizeof	byte alignment	machine type
char	1	1	byte
short	2	2	halfword
int	4	4	word
enum	4	4	word
float	4	4	word
pointer	8	8	doubleword
long int	8	8	doubleword
double	8	8	doubleword
long long	8	8	doubleword

Table 2.1: Mapping from C types to machine data types. A machine type must be aligned to an address which is a multiple of the type's size.


```
struct {  
    char c;  
    short s;  
    long l;  
    double d;  
};
```

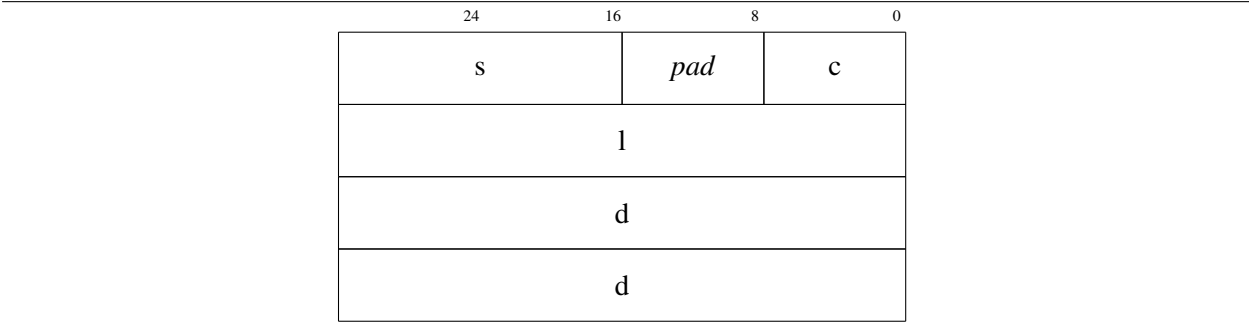


Figure 2.2: Each primitive type is aligned to a multiple of its size. Internal padding is added to assure alignment.

2.2.3 Aggregates and Unions

As seen in Table 2.1, the Tile Processor Architecture requires that all primitive data types be aligned to addresses that are integral multiples of their size. Consequently, aggregate types (structures and arrays) must be carefully arranged in order meet the alignment requirements. An aggregate or union must be aligned in the same way as its largest component primitive type (including components in nested structures), and it must be padded to a size that is a multiple of the alignment.

The following examples illustrate the alignment requirements.

2.2.3.1 Bit Fields

When arranging bitfields within a structure, the compiler should satisfy the following rules. First, each bitfield must fit within a region with the size and alignment of its storage quantifier. Thus, a bitfield declared with a “short” storage quantifier must fit entirely within an aligned, halfword region. Second, the bits should be located at the lowest possible bit address such that they come after any previously declared fields and satisfy condition one. Thus, the “storage region” may overlap, but the bit locations must not interfere with any previously declared data. The following figures illustrate the use of these rules. Third, aggregates and unions must still be aligned and padded according to the size of their largest member.

```
struct{
    double d;
    char c;
}
```

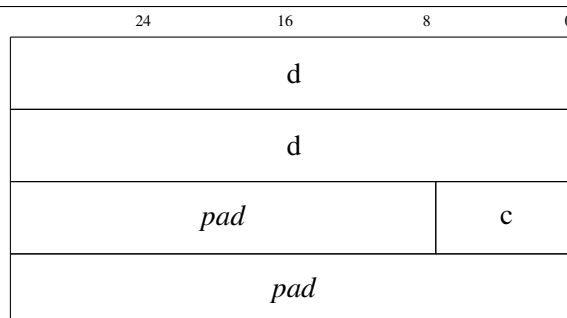


Figure 2.3: Structures must be padded at the end so that their size is a multiple of the largest alignment requirement.

```
struct{
    char s1;
    char s2;
}
```

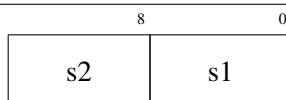


Figure 2.4: This structure has a size of two bytes, but it need only be aligned to one byte because its smallest primitive has an alignment requirement of one byte. Thus, the structure could be allocated at even or odd byte addresses.

```
struct{
    char b:3;
    char c:4;
};
```

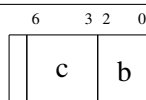


Figure 2.5: Multiple bitfields may be compressed into a single storage region.

```
struct{
  short  b:10;
  short  c:7;
};
```

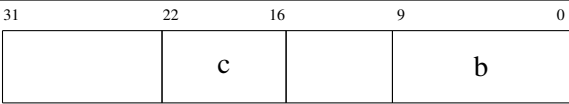


Figure 2.6: If a bitfield cannot fit in overlapped storage without overwriting previously allocated bits, it must be allocated at the next aligned location. This structure must be stored with halfword alignment.

```
struct {
  int  a:20;
  double b;
  int c:2;
};
```

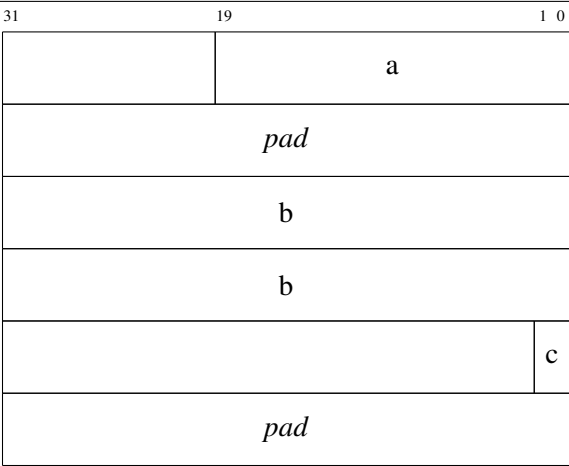


Figure 2.7: Fields must be properly aligned, and structures must be post-padded to a size that is a multiple of their alignment.

Register	Assembler name	Type	Purpose
0 - 9	r0 - r9	Caller-saved	Parameter passing / return values
10 - 29	r10 - r29	Caller-saved	
30 - 51	r30 - r51	Callee-saved	
52	r52	Callee-saved	optional frame pointer
53	tp	Dedicated	Thread-local data
54	sp	Dedicated	Stack pointer
55	lr	Caller-saved	Return address
56	sn		Always zero
57	idn0	Network	IO dynamic network 0
58	idn1	Network	IO dynamic network 1
59	udn0	Network	User dynamic network 0
60	udn1	Network	User dynamic network 1
61	udn2	Network	User dynamic network 2
62	udn3	Network	User dynamic network 3
63	zero		Always zero

Table 2.2: Register assignments for the Tile Processor ABI

2.3 Function Call Specification

This section defines the conventions to be used when a program makes function calls on the Tile Processor Architecture. These conventions are designed to support C-style function calls while enabling backtracing for debugging purposes.

2.3.1 Register Usage

Table 2.2 defines the register conventions used by ABI compliant programs. Registers may be designated as dedicated, caller-saved, callee-saved or network. Caller-saved registers may be used for any purpose and may change value after a function call is made, thus the caller must save them if it wants to preserve their value. Callee-saved registers must have the same value when a function returns as when it was entered.

Dedicated registers are reserved for a particular data item, such as a stack pointer. ABI compliant programs should use dedicated registers as specified; even temporary usage of a dedicated register for another purpose may lead to exposed, inconsistent state if a trap occurs. Dedicated registers are always callee-saved.

Network registers correspond to hardware FIFOs and are not relevant to the calling convention.

2.3.2 Stack

Unlike some other architectures, the TILE-Gx Processor does not include dedicated instructions for stack manipulation. The stack is managed entirely by the software program, which stores a stack pointer in **sp**. The ABI requires that a program's stack grow downward. Thus, the stack pointer starts at a high address

Region	Purpose	Size
Locals	Local variables and register spill slots	Variable
Dynamic space	Dynamic stack space (e.g. alloca())	Variable
Argument space	Callee arguments beyond first 10 words	Variable
Frame pointer	Caller space to spill incoming sp	One word
Callee lr	Callee space to spill incoming lr	One word

Table 2.3: The regions in a stack frame, with locals at the highest address and **lr** spill space at the lowest. On entry to a function the stack pointer points to the callee **lr** spill location set up by the caller.

and decreases as stack frames are pushed on by decrementing the stack pointer. The stack pointer is always aligned to a doubleword (64-bit) boundary.

A stack frame is divided into several regions:

Locals: The function may allocate this frame space for any required local variables, temporaries or register spill targets, etc.

Dynamic space: This region contains memory whose size cannot be known statically, e.g. **alloca()** memory, variable-length arrays, etc. As memory is dynamically allocated, this region grows and the following regions are effectively “slid over” to make room for it. If dynamic memory is allocated, the offset from **sp** to the Locals region is no longer known, so the function copies its initial **sp** value to **r52** and uses that to access the Locals.

Argument space: When a subroutine is called, if it requires more arguments than fit in ten registers then the calling routine must pass those excess arguments by storing them here. Arguments are stored with the first argument at the lowest address, according to the argument passing convention described below. This region holds the maximum argument space needed by *any* call in the owning function, so the total stack frame size can be determined at compile time.

Frame pointer: To assist backtracing, a function must store its own frame pointer here before calling any subroutines. The frame pointer is the value of **sp** on entry to the caller. Calleees are not allowed to modify this memory, so the caller can store its frame pointer here once and then make many calls. Leaf functions, by definition, do not need to store anything here.

Callee **lr:** Non-leaf functions, as well as those that modify **lr** by using it as a general register, must store their incoming **lr** value here. The backtracer expects to see the instruction **sw sp, lr** perform the store. Functions that do not modify **lr** (including through a **jal**) can ignore this memory location.

2.3.2.1 Allocating a stack frame

There are two different ways to allocate a stack frame.

Functions with fixed-size frames less than 32K in size should use a single **addi sp, sp, -N** or **addli sp, sp, -N** instruction to allocate the frame, and a single **addi sp, sp, N** or **addli sp, sp, N** to deallocate it. The backtracer looks for these specific instructions to determine if a PC is in the prolog or epilog.

If this technique cannot be used (i.e., the frame size is not statically known, or is too large for a single **addli**), then the function must set up an explicit frame pointer with **move r52, sp** in the prolog, and only decrement the stack pointer using some instruction other than **addi** or **addli**, such as **sub**.

2.3.3 Argument Passing

The first ten words of arguments are passed in **r0** through **r9**.

Any arguments beyond that are passed in the “argument space” region of the caller’s stack frame, meaning the receiving function will find them at **sp + 16** on entry.

No parameter is passed partially in registers and partially on the stack. If a struct parameter will not fit in the remaining registers, it is passed entirely on the stack, and those remaining registers go unused.

If a function returns a struct too large to fit in return registers, the caller passes a pointer to that struct in **r0**, appropriately “sliding over” the other parameters to make room. The “sliding over” process is performed just as if the function took an extra pointer value as its first parameter, so all of the alignment and other constraints listed above are properly maintained.

As this is a little endian processor, the least-significant half of a doubleword value is always passed in the lowest-numbered register or stack address.

2.3.4 Variadic Functions

Arguments to variadic functions (those taking “...”) should be passed using the standard calling convention.

However, arguments passed in “...” should be fully-promoted as per the usual C promotion rules. Specifically, the caller must convert byte and halfword integers to word-size integers and convert single-precision **float** values to **double**.

2.3.5 Return Values

A function returns a value in **r0** through **r9**, just as if it were passing that value as the first argument to a function.

If the returned value cannot fit in these registers, it is returned indirectly through a special pointer passed to that function in **r0** as described earlier.

2.4 Binary Image Format

Tile Processor binaries are distributed and loaded in the standard ELF64 format.

Chapter 3

System Interface

3.1 System Calls

System calls are invoked by setting up the argument registers as usual for a subroutine call, storing the syscall number in **r10**, and then executing a **swint1** instruction.